

File handling in python

File handling in Python is a fundamental aspect of programming that allows you to read from and write to files.

Advantages of File Handling in Python

Versatility:

File handling in Python allows you to perform a wide range of operations, such as creating, reading, writing, appending, renaming, and deleting files.

Flexibility:

File handling in Python is highly flexible, as it allows you to work with different file types (e.g. text files, binary files, CSV files, etc.), and to perform different operations on files (e.g. read, write, append, etc.). *Executable files, compiled programs, SAS and SPSS system files, spreadsheets, compressed files, and graphic (image) files-binary file

User-friendly:

Python provides a user-friendly interface for file handling, making it easy to create, read, and manipulate files.

Cross-platform:

Python file-handling functions work across different platforms (e.g. Windows, Mac, Linux), allowing for seamless integration and compatibility.

Opening a File:

You can open a file using the `open()` function. It takes two arguments - the file name and the mode (read, write, append, etc.).

```
In [28]: # Open a file for reading
file = open('C:/Users/HP/Desktop/DSML/first example1.txt', 'r')

# Open a file for writing
file = open('C:/Users/HP/Desktop/DSML/first example1.txt', 'w')

# Open a file for appending
file = open('C:/Users/HP/Desktop/DSML/first example1.txt', 'a')
```

```
In [29]: file
```

```
Out[29]: <_io.TextIOWrapper name='C:/Users/HP/Desktop/DSML/first example1.txt' mode='a' encoding='cp1252'>
```

Reading from a File:

There are different methods to read from a file:

`read()`: Reads the entire file.

```
In [32]: # Open a file for reading
file = open('C:/Users/HP/Desktop/DSML/first example1.txt', 'r')

print(file.read())
```

India is my country, and all Indians are my brothers and sisters.
I love my country, and I am proud of its rich and varied heritage.
I shall always strive to be worthy of it.
I shall give my parents, teachers, and all elders respect and treat everyone with courtesy.

```
In [33]: file = open('C:/Users/HP/Desktop/DSML/first example1.txt', 'r')

file.readline()
```

```
Out[33]: 'India is my country, and all Indians are my brothers and sisters. \n'
```

```
In [34]: file.readline()
```

```
Out[34]: 'I love my country, and I am proud of its rich and varied heritage.\n'
```

```
In [35]: file = open('C:/Users/HP/Desktop/DSML/first example1.txt', 'r')
line1 = file.readline()
line2 = file.readline()
print(line1)
print(line2)
```

India is my country, and all Indians are my brothers and sisters.
I love my country, and I am proud of its rich and varied heritage.

```
In [36]: file = open('C:/Users/HP/Desktop/DSML/first example1.txt', 'r')

lines = file.readlines()
for line in lines:
    print(line)
```

India is my country, and all Indians are my brothers and sisters.
I love my country, and I am proud of its rich and varied heritage.
I shall always strive to be worthy of it.
I shall give my parents, teachers, and all elders respect and treat everyone with courtesy.

Writing to a File:

To write to a file, use the write() method.

"w" (write):

1. Opens the file for writing.
2. If the file already exists, it truncates (clears) the content.
3. If the file does not exist, it creates a new file.

```
In [37]: file = open('example.txt', 'w')
file.write('Hello, this is a sample text.')
file.close()
```

"a" (append):

1. Opens the file for writing.
2. If the file already exists, it appends new content to the end without truncating the existing content.
3. If the file does not exist, it creates a new file.

```
In [41]: file = open('C:/Users/HP/Desktop/DSML/first example1.txt', 'a')
file.write('Hello, this is a sample text.\n')
file.close()
```

```
In [42]: file = open('C:/Users/HP/Desktop/DSML/first example1.txt', 'r')

print(file.read())
```

```
India is my country, and all Indians are my brothers and sisters.
I love my country, and I am proud of its rich and varied heritage.
I shall always strive to be worthy of it.
I shall give my parents, teachers, and all elders respect and treat everyone with courtesy.
Hello, this is a sample text.
Hello, this is a sample text.
```

```
In [43]: # writing multiple lines
```

```
In [44]: lines_to_write = ['Line 1\n', 'Line 2\n', 'Line 3\n']

file= open('C:/Users/HP/Desktop/DSML/first example1.txt', 'a') #what if it is "a"?
file.writelines(lines_to_write)
```

```
In [46]: file = open('C:/Users/HP/Desktop/DSML/first example1.txt', 'r')

print(file.read())
```

India is my country, and all Indians are my brothers and sisters.
I love my country, and I am proud of its rich and varied heritage.
I shall always strive to be worthy of it.
I shall give my parents, teachers, and all elders respect and treat everyone with courtesy. Hello, this is a sample text. Hello, this is a sample text.
Line 1
Line 2
Line 3

with statement

working with files, the with statement provides a convenient way to open, read or write to the file, and automatically close the file when the block of code is exited.

```
In [47]: with open('C:/Users/HP/Desktop/DSML/first example1.txt', 'r') as file:
         content = file.read()
         print(content)
```

India is my country, and all Indians are my brothers and sisters.
I love my country, and I am proud of its rich and varied heritage.
I shall always strive to be worthy of it.
I shall give my parents, teachers, and all elders respect and treat everyone with courtesy. Hello, this is a sample text. Hello, this is a sample text.
Line 1
Line 2
Line 3

exception handling

Error vs exception

An error is an issue in a program that prevents the program from completing its task.

In comparison, an exception is a condition that interrupts the normal flow of the program. Both errors and exceptions are a type of runtime error, which means they occur during the execution of a program.

```
In [48]: # Example of a ZeroDivisionError
         # Attempting to divide a number by zero
         result = 10 / 0
         result
```

```
-----
ZeroDivisionError                                Traceback (most recent call last)
~\AppData\Local\Temp\ipykernel_5396\726266597.py in <module>
      1 # Example of a ZeroDivisionError
      2 # Attempting to divide a number by zero
----> 3 result = 10 / 0
      4 result

ZeroDivisionError: division by zero
```

```
In [49]: # Example of an IOError
# Attempting to open a non-existent file for reading
with open("nonexistent_file.txt", "r") as file:
    content = file.read()
```

```
-----
FileNotFoundError                                Traceback (most recent call last)
~\AppData\Local\Temp\ipykernel_5396\2569326585.py in <module>
      1 # Example of an IOError
      2 # Attempting to open a non-existent file for reading
----> 3 with open("nonexistent_file.txt", "r") as file:
      4     content = file.read()

FileNotFoundError: [Errno 2] No such file or directory: 'nonexistent_file.txt'
```

```
In [50]: # Example of an ImportError
# Trying to import a non-existent module
import non_existent_module
```

```
-----
ModuleNotFoundError                                Traceback (most recent call last)
~\AppData\Local\Temp\ipykernel_5396\2277421959.py in <module>
      1 # Example of an ImportError
      2 # Trying to import a non-existent module
----> 3 import non_existent_module

ModuleNotFoundError: No module named 'non_existent_module'
```

```
In [51]: # Example of an AttributeError
# Accessing a non-existent attribute or method of an object
my_string = "Hello, World!"
print(my_string.uppercase())
```

```
-----
AttributeError                                Traceback (most recent call last)
~\AppData\Local\Temp\ipykernel_5396\3433423019.py in <module>
      2 # Accessing a non-existent attribute or method of an object
      3 my_string = "Hello, World!"
----> 4 print(my_string.uppercase())

AttributeError: 'str' object has no attribute 'uppercase'
```

```
In [52]: # Example of a ValueError
# Trying to convert an invalid string to an integer
invalid_number = int("abc")
```

```
-----
ValueError                                Traceback (most recent call last)
~\AppData\Local\Temp\ipykernel_5396\1939790458.py in <module>
      1 # Example of a ValueError
      2 # Trying to convert an invalid string to an integer
----> 3 invalid_number = int("abc")

ValueError: invalid literal for int() with base 10: 'abc'
```

```
In [53]: # Example of an IndexError
# Accessing an element at an index that doesn't exist
my_list = [1, 2, 3]
print(my_list[5])
```

```
-----
IndexError                                Traceback (most recent call last)
~\AppData\Local\Temp\ipykernel_5396\4285615035.py in <module>
      2 # Accessing an element at an index that doesn't exist
      3 my_list = [1, 2, 3]
----> 4 print(my_list[5])

IndexError: list index out of range
```

exceptions can interrupt the normal flow of the code when they occur.

When an exception is raised, the program jumps out of the normal execution sequence and looks for an exception handler to process the exception.

how can we handle exceptions?

```
In [54]: print(x)
```

```
-----
NameError                                Traceback (most recent call last)
~\AppData\Local\Temp\ipykernel_5396\1353120783.py in <module>
----> 1 print(x)

NameError: name 'x' is not defined
```

```
In [55]: try:
          print(x)
        except:
            print("An exception occurred") #indentation error is not considered an exception i
                                           #Indentation errors occur when there are issues with t
                                           #particularly related to the indentation level.
```

An exception occurred

```
In [56]: #Print one message if the try block raises a NameError and another for other errors:
try:
    print(x)
except NameError:
    print("Variable x is not defined")
except:
    print("Something else went wrong")
```

Variable x is not defined

```
In [57]: try:
        print(x)
    except ValueError:
        print("Variable x is not defined")
    except:
        print("Something else went wrong") #its a name e
```

Something else went wrong

```
In [58]: #In this example, the try block does not generate any error:
```

```
try:
    print("Hello")
except:
    print("Something went wrong")
else:
    print("Nothing went wrong")
```

Hello

Nothing went wrong

```
In [59]: #finally
try:
    print(x)
except:
    print("Something went wrong")
finally:
    print("The 'try except' is finished")
```

Something went wrong

The 'try except' is finished

1. The try block lets you test a block of code for errors.

2.The except block lets you handle the error.

3.The else block lets you execute code when there is no error.

4.The finally block lets you execute code, regardless of the result of the try- and except blocks.

```
In [26]: try:
        # This code will be executed
        result = 10 / 0 # This raises a ZeroDivisionError
        print("This line won't be reached")
    except ZeroDivisionError:
        # The program jumps here when a ZeroDivisionError occurs
        print("Error: Division by zero.")
    else:
        # This block will not be executed because of the exception
        print("No exceptions occurred.")
    finally:
        # This block always executes, regardless of exceptions
        print("This block always executes.")
```

Error: Division by zero.

This block always executes.

```
In [60]: try:
# This code will be executed
result = 10 / 2 # This not raises a ZeroDivisionError
print("This line won't be reached")
except ZeroDivisionError:
# The program jumps here when a ZeroDivisionError occurs
print("Error: Division by zero.")
else:
# This block will not be executed because of the exception
print("No exceptions occurred.")
finally:
# This block always executes, regardless of exceptions
print("This block always executes.")
```

This line won't be reached
No exceptions occurred.
This block always executes.

In []:

In []:

In []:

In []:

In []: